

VISIONA Web Application Security Assessment

Complete preliminary assessment report — Burp Suite Community Edition

Target: <https://mywebpgehtml.vercel.app/>

Assessment date: 7 October 2026

Assessment type: Preliminary web application security assessment

Tester: Project author

1. Executive Summary

I performed a preliminary security assessment of the VISIONA web application, which is my own web project deployed on Vercel. The purpose of the assessment was to understand how the application communicates over HTTP and to identify practical security-hardening opportunities. Burp Suite Community Edition was used to intercept and review requests and responses, while the returned HTML was also inspected for client-side behavior.

The assessment confirmed that the site is using HTTPS and returning an HSTS policy. It also identified several areas that should be reviewed or improved, including the absence of some common security headers, a permissive CORS response, and the use of inline JavaScript. These observations are documented as security-hardening items rather than being presented as confirmed critical vulnerabilities.

2. Scope

Testing was limited to the VISIONA application and its publicly accessible web resources. The assessment did not target Vercel infrastructure, third-party systems, or services outside the application's control. Testing performed in this report was limited to manual observation and low-impact web security checks.

3. Tools and Methodology

Tool / Method	Purpose
Burp Suite Community Edition	Intercept and review HTTP requests and responses
Burp Proxy / HTTP History	Capture application traffic and identify accessible resources
HTTP response-header review	Check HTTPS and browser security controls
HTML / JavaScript inspection	Review client-side code and script loading behavior
Browser testing	Confirm normal application behavior while traffic is captured

4. Assessment Results

Area	Observation	Assessment
HTTPS / HSTS	Strict-Transport-Security was returned with max-age=63072000; includeSubDomains; preload.	Positive
Content Security Policy	No Content-Security-Policy header was observed in the captured response.	Hardening opportunity
X-Content-Type-Options	No X-Content-Type-Options header was observed in the captured response.	Hardening opportunity
CORS	Access-Control-Allow-Origin: * was observed.	Requires context/review
Server disclosure	The response included Server: Vercel.	Informational
Client-side JavaScript	Inline JavaScript was observed in the HTML response.	Relevant to CSP design

5. Detailed Findings and Recommendations

5.1 Missing Content Security Policy

Observation: A Content-Security-Policy header was not observed in the captured response. CSP is a browser security control that can reduce the impact of certain client-side attacks when correctly configured.

Recommendation: VISIONA contains inline JavaScript, so a CSP should not be copied blindly. The policy should be designed around the application's actual scripts and resources. After deployment, the site should be retested to confirm both security and functionality.

5.2 Missing X-Content-Type-Options

Observation: The captured response did not include X-Content-Type-Options. This is a common browser security-hardening header.

Recommendation: Configure X-Content-Type-Options: nosniff where appropriate, redeploy the application, and verify the header in a fresh Burp response.

5.3 Permissive CORS Response

Observation: Access-Control-Allow-Origin: * was observed. A wildcard CORS response is not automatically a vulnerability; its impact depends on what resources the application exposes and whether sensitive data is available cross-origin.

Recommendation: Review whether cross-origin access is actually required. If it is not required, remove the permissive policy. If it is required, restrict access to trusted origins and verify credential handling.

5.4 Server Technology Disclosure

Observation: The response included Server: Vercel. This provides minor information about the hosting platform.

Recommendation: Treat this as informational/low risk. Removing platform disclosure is optional and should be weighed against the hosting platform's normal behavior.

5.5 Inline JavaScript

Observation: Inline JavaScript was observed in the HTML response, including application navigation and cart-related logic.

Recommendation: This is not a vulnerability by itself. It is important when creating a CSP because an overly strict policy could break legitimate functionality.

6. Positive Security Control

The application was observed enforcing HTTPS through Strict-Transport-Security. The captured value was: max-age=63072000; includeSubDomains; preload. This is a positive security control and was recorded as part of the assessment.

7. Evidence — Burp Suite Screenshots

The following screenshots are the original assessment screenshots without added red boxes or artificial annotations. They document the traffic capture, response headers, and client-side JavaScript inspection performed during the assessment.

The screenshot displays the Burp Suite interface. The top menu bar includes options like Burp, Project, Intruder, Repeater, View, and Help. Below the menu, there are tabs for Dashboard, Target, Proxy (selected), Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Extensions, Organizer, and Discover. The main window is divided into several sections:

- HTTP history:** A table showing captured requests. The selected request (number 6) is a GET request to `https://mywebpgehtml.vercel...` with a status code of 200 and a MIME type of HTML.
- Request/Response:** A section showing the details of the selected request. The 'Request' tab is active, displaying the raw HTTP request in a text editor. The request includes headers such as `Host: mywebpgehtml.vercel.app`, `Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"`, `Sec-Ch-Ua-Mobile: ?0`, `Sec-Ch-Ua-Platform: "Windows"`, `Accept-Language: en-US,en;q=0.9`, `Upgrade-Insecure-Requests: 1`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36`, and `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=h3;q=0.7`.
- Inspector:** A section on the right side of the interface, currently showing 'Request attributes' with a value of 2. It also has tabs for 'Request headers' (17) and 'Response headers' (14).

The bottom status bar indicates 'Event log (1)', 'All issues', and 'Memory: 154.8MB of 7.87GB'.

Figure 1 — Burp Suite HTTP History and captured VISIONA request

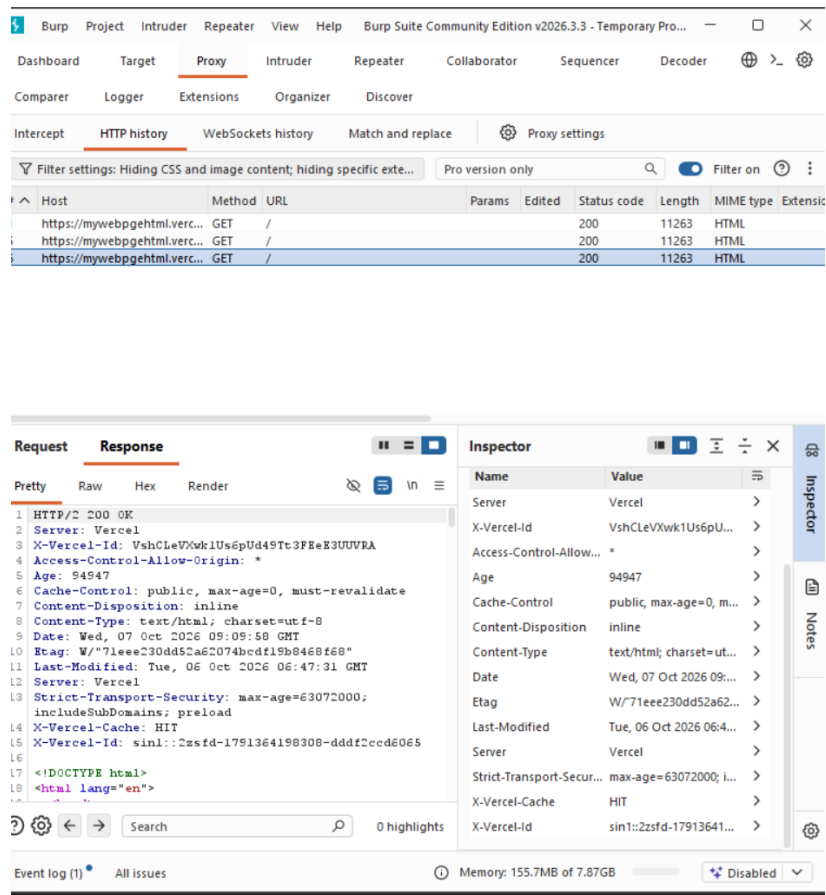


Figure 2 — HTTP response headers captured from the VISIONA deployment

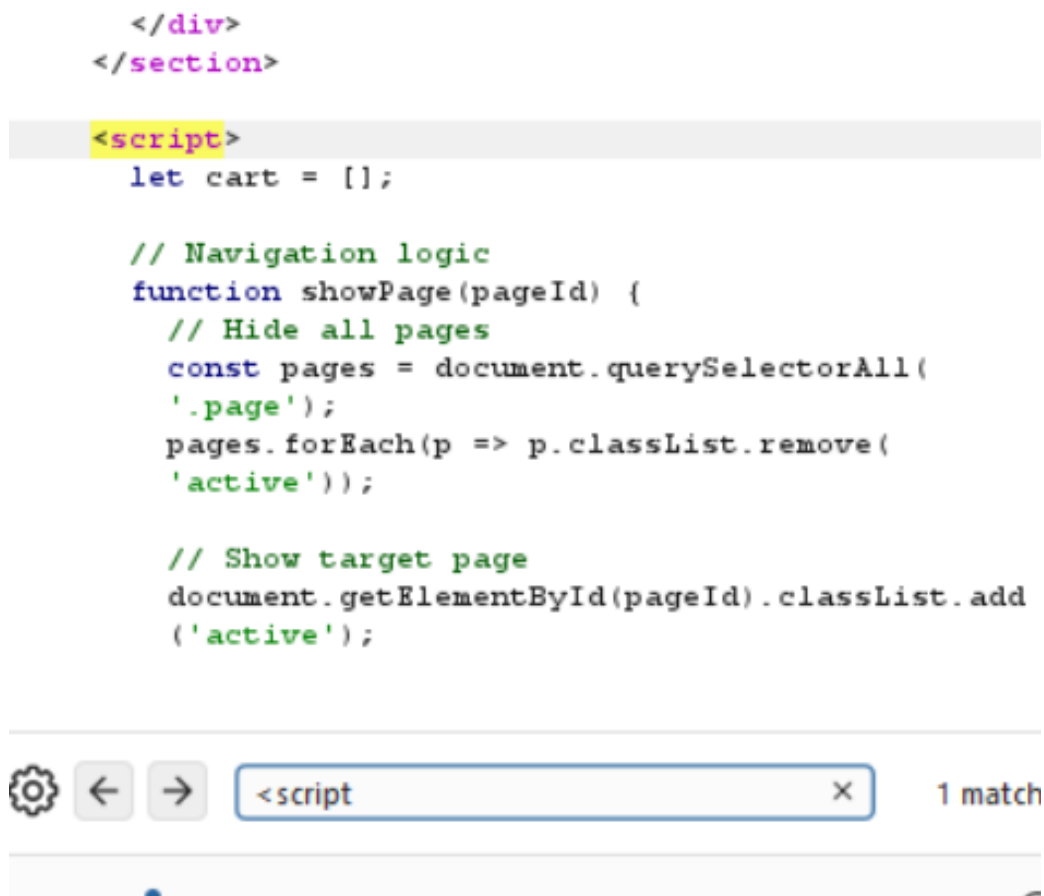


Figure 3 — Inline JavaScript observed in the VISIONA HTML response

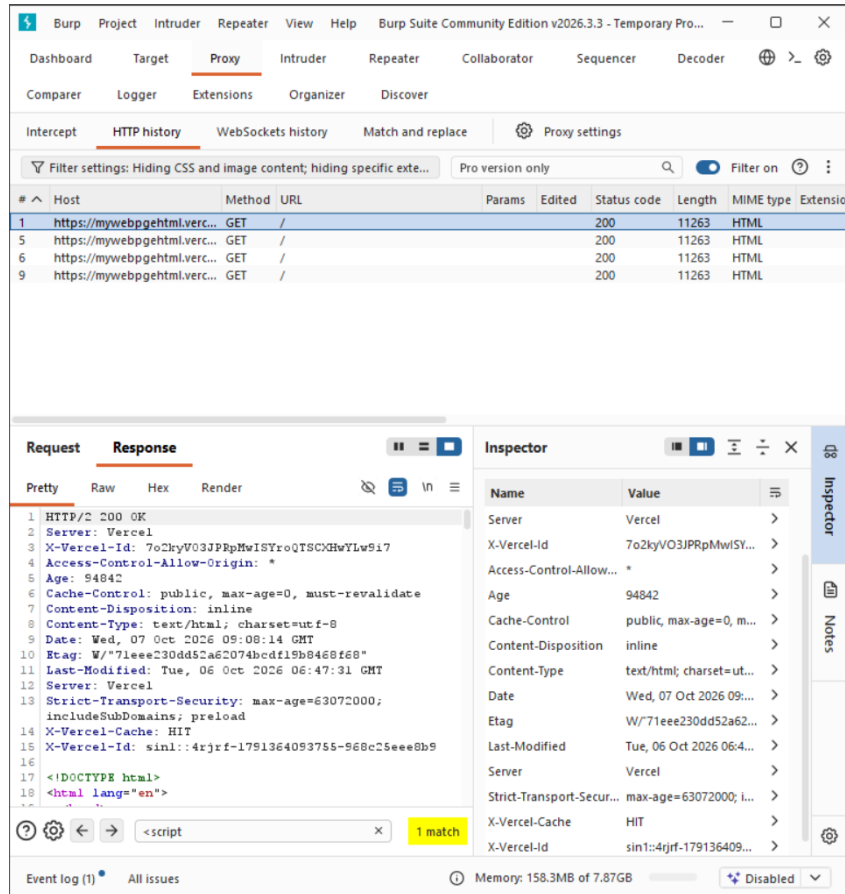


Figure 4 — Follow-up Burp response captured during the assessment

8. Limitations

This report is a preliminary assessment and is not a full penetration test. The work completed so far focuses on HTTP traffic, response headers, and client-side inspection. No claim is made that VISIONA is free of vulnerabilities. Additional controlled testing of forms, inputs, dependencies, and application functionality would be required for a broader assessment.

9. Recommended Next Steps

- Add and verify appropriate security headers.
- Review the wildcard CORS policy and restrict it if cross-origin access is not required.
- Design a CSP that works with the application's current JavaScript and resources.
- Redeploy the application after remediation.
- Repeat the Burp Suite checks and document the before-and-after results.
- Keep the final report and screenshots in the GitHub project as security-assessment evidence.

10. Conclusion

The assessment demonstrated a practical workflow for reviewing a web application with Burp Suite. The VISIONA deployment has HTTPS/HSTS enabled, while several areas of browser security configuration can be improved. The next stage should focus on remediation followed by a fresh Burp Suite retest so that the project can demonstrate a complete security cycle: identify, document, fix, and verify.